

Q1

A robot has two battery backup lines. The first line is checked from the beginning, and the second line is checked from the end. Create a new array by adding the values from opposite directions.

If any value is null, treat it as 0.

Method Name: mergeBatteryBackup

Sample Input:	Sample Output:	Explanation:
batteryA = [6, null, 4, -2, null] batteryB = [3, 8, null, 5, 7]	[13, 5, 4, 6, 3]	$6 + 7 = 13$ $\text{null} + 5 = 5$ $4 + \text{null} = 4$ $-2 + 8 = 6$ $\text{null} + 3 = 3$

Q2

A library stores book counts in two shelves. Shelf 1 is counted from left to right. Shelf 2 is counted from right to left. Add the opposite-position values and return the new array.

Empty shelf positions are stored as null, and they should be counted as 0.

Method Name: mergeShelves

Sample Input:	Sample Output:	Explanation:
shelf1 = [12, 5, null, 9] shelf2 = [4, null, 7, 3]	[15, 12, 0, 13]	$12 + 3 = 15$ $5 + 7 = 12$ $\text{null} + \text{null} = 0$ $9 + 4 = 13$

Q3

A bus counter stores ticket IDs in a fixed-size array. A ticket ID t is marked as damaged. Starting from the first occurrence of t , remove every alternate damaged ticket.

After removing, shift the remaining ticket IDs left and fill empty positions with 0.

Method Name: `removeDamagedTickets`

Sample Input:	Sample Output:	Explanation:
<pre> tickets = [4, 9, 4, 2, 4, 7, 4, 0] t = 4 </pre>	<pre> [9, 4, 2, 7, 4, 0, 0, 0] </pre>	Occurrences of 4: 4_0, 4_1, 4_2, 4_3 Remove 4_0 and 4_2. Keep 4_1 and 4_3.

Q4

A shop stores coupon codes in an array. A coupon code t is selected for cleanup. Remove alternate occurrences of that coupon, starting from the first one.

Keep the order of all other coupons unchanged. Empty spaces should become 0.

Method Name: `cleanCoupons`

Sample Input:	Sample Output:	Explanation:
<pre> coupons = [5, 1, 5, 5, 3, 5, 2, 0] t = 5 </pre>	<pre> [1, 5, 3, 5, 2, 0, 0, 0] </pre>	Occurrences of 5: 5_0, 5_1, 5_2, 5_3 Remove 5_0 and 5_2. Keep 5_1 and 5_3.

Q5

A teacher stores student attendance IDs in an array. Some absent students are marked with 0. Move all present student IDs to the left side while keeping their original order. Fill the remaining positions with 0.

Method Name: shiftPresentStudents

Sample Input: students = [101, 0, 203, 0, 305, 406, 0]	Sample Output: [101, 203, 305, 406, 0, 0, 0]	Explanation: Present IDs are 101, 203, 305, and 406. They are shifted left in the same order.
--	--	---

Q6

A hospital queue is stored in a fixed-size array. Empty positions are marked with 0. A new emergency patient ID must be inserted at a given index. Shift the existing patients one step right.

Assume there is enough empty space at the end.

Method Name: insertPatient

Sample Input: queue = [11, 22, 33, 44, 0, 0] size = 4 patientID = 99 index = 2	Sample Output: [11, 22, 99, 33, 44, 0]	Explanation: 99 is inserted at index 2. 33 and 44 shift one step right.
---	--	---

Q7

An online shop stores order IDs in a fixed-size array. Remove the order at a given index. Shift all later orders one step left. Put 0 at the last used position.

Method Name: removeOrder

Sample Input:

```
orders = [501, 502,
503, 504, 505, 0]
size = 5
index = 1
```

Sample Output:

```
[501, 503, 504, 505,
0, 0]
```

Explanation:

Order 502 was at index 1. After removing it, the right-side orders shift left.

Q8

A delivery company stores riders in a queue. After one round, the first rider goes to the end of the queue, and everyone else shifts one step left.

Method Name: rotateRidersLeft

Sample Input:

```
riders = [7, 14, 21,
28, 35]
```

Sample Output:

```
[14, 21, 28, 35, 7]
```

Explanation:

First rider 7 moves to the last position. All other riders shift one step left.

Q9

A game lobby stores player IDs in an array. Before the match starts, the last player becomes the first player, and everyone else shifts one step right.

Method Name: rotatePlayersRight

Sample Input:

players = [10, 20,
30, 40, 50]

Sample Output:

[50, 10, 20, 30, 40]

Explanation:

The last player moves
50 to index 0.

All other players
shift one step right.

Q10

A delivery office stores parcel IDs in the order they arrived. For return processing, the parcels must be arranged in reverse order in a new array.

Do not change the original array.

Method Name: reverseParcels

Sample Input:

parcels = [13, 26,
39, 52, 65]

Sample Output:

[65, 52, 39, 26, 13]

Explanation:

The first parcel
becomes last.

The last parcel
becomes first.

A new reversed array
is returned.

Q11

```
for (int i = 0; i < n; i++) {
    System.out.println(i);
}
```

What is the time complexity?

- | | | | |
|-----------|-----------|-------------|----------------|
| A. $O(1)$ | B. $O(n)$ | C. $O(n^2)$ | D. $O(\log n)$ |
|-----------|-----------|-------------|----------------|

```
int a = 10;
int b = 20;
int sum = a + b;
System.out.println(sum);
```

What is the time complexity?

- | | | | |
|-----------|-----------|-------------|----------------|
| A. $O(1)$ | B. $O(n)$ | C. $O(n^2)$ | D. $O(\log n)$ |
|-----------|-----------|-------------|----------------|

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.println(i + " " + j);
    }
}
```

What is the time complexity?

- | | | | |
|-----------|-----------|-------------|----------------|
| A. $O(1)$ | B. $O(n)$ | C. $O(n^2)$ | D. $O(\log n)$ |
|-----------|-----------|-------------|----------------|

```
int[] arr2 = new int[n];

for (int i = 0; i < n; i++) {
    arr2[i] = i;
}
```

What is the space complexity?

- | | | | |
|-----------|-----------|-------------|----------------|
| A. $O(1)$ | B. $O(n)$ | C. $O(n^2)$ | D. $O(\log n)$ |
|-----------|-----------|-------------|----------------|

```
int sum = 0;

for (int i = 0; i < n; i++) {
    sum = sum + i;
}
```

What is the space complexity?

- | | | | |
|-----------|-----------|-------------|----------------|
| A. $O(1)$ | B. $O(n)$ | C. $O(n^2)$ | D. $O(\log n)$ |
|-----------|-----------|-------------|----------------|